Architectural Blueprint for a Unified, AI-Native Semantic Filesystem

The Crisis of Infrastructure Complexity in High-Performance Computing

The landscape of high-performance computing (HPC) and artificial intelligence (AI) infrastructures has historically been dominated by parallel filesystems such as Lustre, GPFS, and BeeGFS, operating alongside highly scalable, distributed object stores like Amazon S3 and specialized distributed databases.[1] While these traditional architectures excel at delivering raw storage capacity and sequential throughput, they increasingly fall short when attempting to address the highly dynamic, heterogeneous, and chaotic demands of modern AI-native workloads.[2] Contemporary machine learning pipelines generate exceptionally erratic I/O patterns: massively parallel random reads from distributed training jobs hitting storage from hundreds of nodes simultaneously, small-block writes for frequent checkpoints across multi-day runs, and iterative workloads that generate immense metadata demands where millions of small files require microsecond latency.[3]

Compounding this severe storage bottleneck is the proliferation of specialized, loosely coupled tooling required to manage enterprise data flows. Modern distributed systems rely on a highly fragmented ecosystem of application programming interfaces (APIs) and discrete services to manage complex data topologies. Developers are forced to deploy Apache Kafka or Redpanda for event streaming and publish/subscribe (pub/sub) messaging [5], Redis for in-memory metadata caching and key-value storage [8], Git for cryptographic versioning and state snapshots, Hadoop Distributed File System (HDFS) for batch analytics, and Elasticsearch for distributed indexing and full-text search. The requirement to orchestrate these disparate tools results in heavy client-side logic, elevated network overhead, and a severe cognitive load on system architects and developers.[10]

A compelling architectural alternative proposes reverting to the fundamental Unix philosophy: "Everything is a file".[10] This approach envisions a novel, distributed filesystem that natively integrates object storage, distributed message queues, pub/sub channels, append-only logs, and ring buffers into a unified, purely POSIX-compliant file interface.[14] By abstracting complex distributed data topologies entirely behind standard read() and write() system calls, and coupling this abstraction with a zero-configuration, AirDrop-like peer discovery mechanism over high-speed Remote Direct Memory Access (RDMA) fabrics [17], the cognitive burden on developers drops effectively to zero. Furthermore, by managing global metadata and file indexes through advanced algorithmic string-matching data structures—such as inverted trees (suffix trees), suffix arrays, Bloom filters, or dynamic Aho-Corasick automata [20]—the filesystem can transform standard directory navigation into a powerful semantic query engine.[23]

This report exhaustively evaluates the design constraints, algorithmic tradeoffs, hardware interactions, and architectural feasibility of constructing such a unified system, specifically tailored for exascale HPC and AI-native applications.

Redefining the System Abstraction: "Everything is a File" at Scale

The concept of treating all system resources as file descriptors originates from early Unix designs and was deeply refined by the Plan 9 operating system via its 9P protocol.[25] In the Plan 9 paradigm, networks, user interface elements, hardware devices, and complex inter-process communications are universally represented as hierarchical file systems, allowing applications to manipulate distributed resources via standard file operations.[27] Extending this paradigm to a modern AI and HPC distributed filesystem requires mapping complex data structures and event-driven architectures directly to the Linux Virtual File System (VFS).[29]

The Virtual File System and FUSE Implementation Constraints

The VFS operates as the primary abstraction layer and dispatch machinery between user-space code (executing POSIX syscalls like open, read, write, close, stat, and mmap) and concrete filesystem implementations.[29] Implementing a novel filesystem traditionally requires highly complex kernel module development. However, the Filesystem in Userspace (FUSE) framework provides a software interface that allows the creation of fully functional virtual filesystems without editing kernel code.[31]

In a FUSE implementation, a user-space daemon registers a handler with the kernel. When an application issues a syscall, the VFS routes it to the FUSE kernel module, which then proxies the request to the user-space handler via a specialized queue.[32] While FUSE allows rapid implementation of virtual directories and custom inode structures (such as mapping a RESTful S3 bucket to a local directory tree) [34], it introduces significant context-switching overhead.[37] FUSE operations involve multiple transitions between user space and kernel space, occupying the FUSE queue and severely limiting the ability to cache certain operations in the kernel.[37] As an architectural constraint, while FUSE is ideal for the semantic and control planes of the proposed filesystem, high-throughput AI data payloads must eventually bypass the VFS entirely via zero-copy RDMA mechanisms, which will be discussed in subsequent sections.

Mapping Data Topologies to Filesystem Semantics

To achieve the operational power of disparate tools like Mofka, Kafka, Redis, and S3 without introducing new APIs, the system must transparently translate POSIX syscalls into native distributed data structure operations. The following table illustrates the proposed mapping between standard file operations and complex distributed system behaviors.

Distributed	Filesystem	POSIX Interaction	Underlying System

Concept	Representation	Mechanism	Behavior
Message Queue	Named Pipe (FIFO)	open(), read(), write()	Data written is appended to a distributed log; reads dynamically advance the consumer offset.[38]
Pub/Sub Channel	Virtual Directory	open(), read() by multiple clients	Scatter-gather distribution of incoming data streams to all open file descriptors listening to the inode.[7]
Object Store	Append-Only File	open(..., O_APPEND), write()	Data buffered in SCM; asynchronously flushed as immutable blocks to cold object storage (S3 API bridge).[34]
Ring Buffer	Circular File (via xattr)	Continuous write()	Block allocator wraps the head pointer to overwrite oldest logical blocks when capacity quota is reached.[45]
Versioning (Git)	Immutable Snapshot	Read-only Virtual Mount	Snapshotting of metadata state mimicking Plan 9's Fossil/Venti architecture.[11]

Integrating Distributed Message Queues and Pub/Sub

Traditional publish/subscribe messaging decouples producers and consumers, avoiding the

performance costs of synchronous RPC calls.[7] In the proposed unified filesystem, a pub/sub topic can be represented natively as a specialized directory or a FIFO managed by the VFS.[38] Producers issue write() commands to the file, and consumers execute read() commands. While standard POSIX message queues implement a localized store-and-forward design [14], a distributed implementation must scale across thousands of nodes.

Inspired by experimental frameworks like KafkaFS, which exposes Kafka clusters directly via FUSE [40], the underlying distributed storage engine intercepts writes to specific inodes and appends them to a distributed, partitioned log. The Mofka framework provides a blueprint for this in HPC environments. Mofka, an event-streaming service built on the Mochi suite of microservices, splits events into a structured JSON metadata part and a raw payload data part.[5] This separation allows the filesystem to optimize data and metadata independently, ensuring that pub/sub messages are distributed at line rate over the network fabric without overwhelming the metadata manager.[6]

Reimagining Ring Buffers and Circular Files

HPC diagnostic applications, kernel logging, and continuous AI telemetry frequently require ring buffers—structures where data overwrites the oldest entries once a predefined capacity threshold is reached.[15] Operating systems utilize this concept natively for kernel logs (e.g., the dmesg utility reading from the kernel ring buffer at /dev/kmsg).[52]

Within the proposed filesystem, there is no need for an external streaming database to maintain sliding windows. A circular file can be instantiated by setting specific extended attributes (e.g., setfattr -n user.type -v ring_buffer /mnt/fs/stream). The underlying block allocator is programmed to maintain a circular linked list of physical storage blocks. When the file reaches its defined quota, the metadata pointer representing the "head" of the file wraps around, seamlessly overwriting the oldest logical block. To the application executing the write() syscall, the file appears as an infinite sink, while the physical disk footprint remains strictly bounded.[15]

Bridging Object Stores with Append-Only Semantics

Object stores (e.g., AWS S3, MinIO) typically rely on RESTful HTTP APIs, which fundamentally clash with POSIX semantics.[43] Tools like S3FS and Goofys attempt to translate POSIX commands into S3 API calls, but they introduce severe latency degradation.[34] Every simple metadata operation, such as rename() or chmod(), requires synchronous network communication, and random writes necessitate downloading, modifying, and re-uploading entire objects.[43]

To seamlessly present a highly scalable object store as a local filesystem without the latency penalty, the architecture must completely decouple metadata from data payloads.[8] When an application opens a file in append mode (O_APPEND), the filesystem engine buffers the incoming data locally in ultra-fast Storage Class Memory (SCM) or NVMe drives. As these buffers fill, the system asynchronously flushes the immutable, large data blocks to the underlying cold object store in the background.[34] The metadata pointers are simultaneously

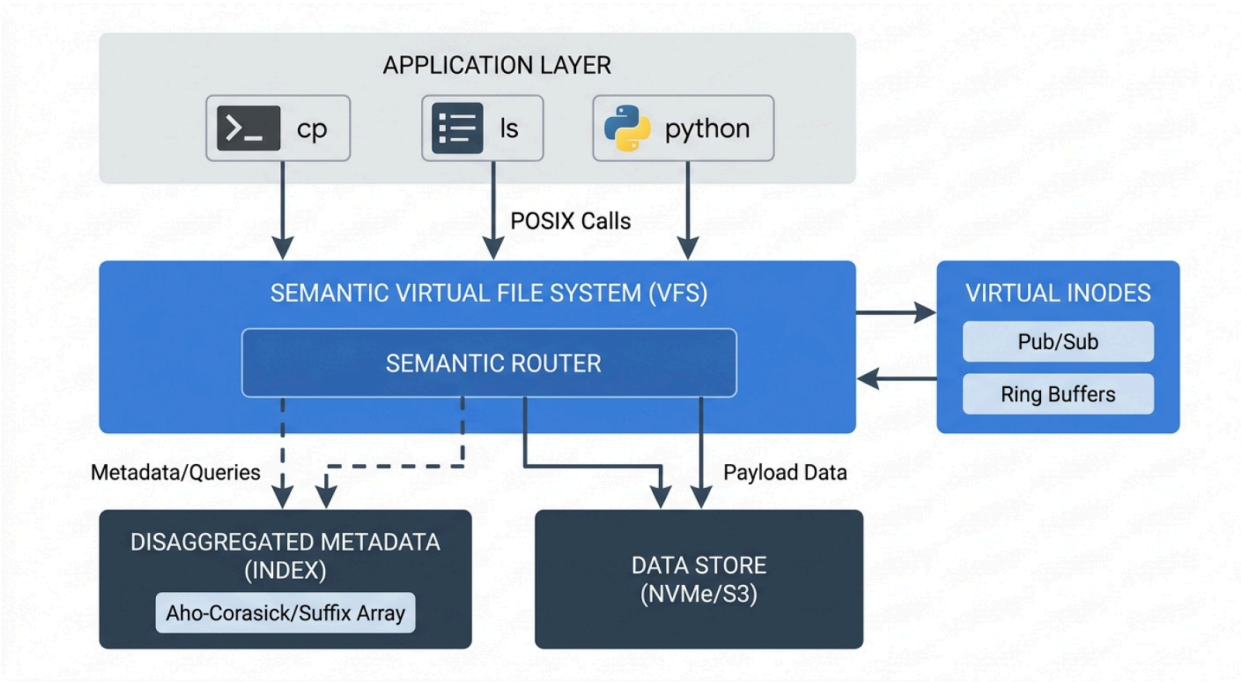
updated in the unified global namespace, ensuring that applications can continue to read the data transparently, regardless of whether it resides in the hot NVMe tier or the cold object tier.[44]

The Semantic File System: Directories as Queries

A pivotal requirement of the user query is the ability to hide complex search APIs (such as those required by Elasticsearch) behind native file operations. The MIT Semantic File System (SFS), introduced in 1991, pioneered this approach by creating "virtual directories," where a directory path logically operates as a conjunctive query.[23]

In a traditional hierarchical filesystem, files are rigidly bound to static directory paths. In a semantic filesystem, the architecture introduces a layer that dynamically generates directories by extracting attributes from files.[60] Users interact with the filesystem's rich metadata layers by applying POSIX extended attributes (xattr), which consist of namespace-qualified key-value pairs (e.g., user.project=AI_model_v2, trusted.status=checkpoint).[62] Furthermore, the system employs file-type-specific transducers—automated daemons that parse ingested files to automatically extract and index metadata.[23]

Instead of deploying and maintaining a separate distributed search cluster, the user navigates the file system directly. A query path such as /mnt/fs/search/author=smith/type=checkpoint/ is intercepted by the VFS. The filesystem's indexing engine parses this path as a boolean AND constraint.[23] The VFS dynamically and instantaneously generates a virtual directory containing symbolic links—or direct inode mappings—to all files across the entire distributed cluster that satisfy the parameters.[23] This elegant abstraction enables highly complex data discovery using standard UNIX tools like ls, grep, or find, entirely bypassing the need for developers to write custom API query scripts.[23]



Applications utilize standard file operations, which the Virtual File System intercepts and translates into distributed messaging, object storage, and semantic index queries, eliminating the need for specialized APIs.

Algorithmic Foundations for Distributed Indexing

To support a semantic filesystem capable of replacing dedicated search tools like Elasticsearch, the underlying metadata indexing engine must be exceptionally fast, infinitely scalable, and highly memory-efficient. The core proposition involves evaluating the use of inverted trees (suffix trees), Aho-Corasick automata, Bloom filters, and other advanced string-matching mechanisms. Each of these algorithmic data structures carries distinct mathematical constraints when scaled to HPC workloads encompassing billions of discrete files and extended attributes.[2]

Suffix Trees and the Memory Explosion Constraint

A suffix tree is a profoundly powerful data structure that indexes all suffixes of a given string in linear $O(n)$ time (commonly utilizing Ukkonen's algorithm) and allows pattern matching in time proportional only to the pattern length $O(m)$.[20] This characteristic makes suffix trees highly attractive for indexing complex metadata, tracking string similarities, and supporting approximate pattern matching across disparate file attributes.[20] Furthermore, suffix trees

possess the unique ability to identify systematic redundancies among suffixes—allowing the system to observe that a subtree rooted at an internal node is isomorphic to subtrees rooted elsewhere—a capability notably absent from standard suffix arrays or FM-indexes.[20]
However, the fatal constraint of suffix trees in a distributed filesystem context is their massive memory footprint. While a raw text corpus can be compactly encoded in $n \log \sigma$ bits, a careful implementation of a suffix tree requires $\Theta(n)$ memory words.[68] If naive construction methods are used—such as building a character tree with a single character at every node—a string will generate $O(n^2)$ nodes, leading to immediate memory exhaustion even for small 10MB inputs.[67] Even highly optimized suffix tree implementations demand a working memory overhead of 30× to 60× the size of the input data.[68]
If an HPC metadata index reaches multiple terabytes in size, a 60× memory expansion makes distributed RAM storage economically and physically unfeasible.[70] Furthermore, distributed suffix tree construction suffers from severe network communication bottlenecks. The scatter-gather overhead of exchanging machine words across cluster nodes scales poorly, even when utilizing techniques like random chunk redistribution, because collective operations like broadcast and prefix sum incur heavy start-up and transmission latencies ($O(\alpha \log p + \beta h)$).[69]

Lightweight Alternatives: Suffix Arrays and FM-Indexes

To mitigate the catastrophic memory crisis associated with suffix trees, the filesystem architecture must pivot to Suffix Arrays or FM-indexes.[20] A suffix array is a lexicographically sorted array of all suffixes of a string, originally introduced by Manber and Myers as a space-efficient alternative that still preserves rapid $O(m \log n)$ search times.[22]
Modern distributed suffix array construction algorithms, such as the lightweight distributed adaptation of the difference cover (DCX/DC3/DC7) algorithm, rely on advanced bucketing techniques.[69] These adaptations effectively reduce working memory requirements down to merely 14× to 26× the input size for medium-sized inputs.[69] While the underlying prefix-doubling and difference cover algorithms remain complex to implement over distributed memory, they offer linear work and polylogarithmic depth, scaling predictably across computing nodes.[72]
Enhanced suffix arrays (ESAs), when paired with auxiliary data tables (like the longest common prefix array), can fully mimic the functionality of suffix trees without the massive pointer overhead.[22] Furthermore, applying the Burrows-Wheeler Transform (BWT) to construct Compressed Suffix Arrays (CSA) or FM-indexes allows the metadata index to consume less physical space than the original text.[68] An FM-index provides a fast worst-case search time of $O(m/\log_{|\Sigma|} n + \log_{|\Sigma|} n)$ while using at most $(1 + o(1))n \log |\Sigma|$ bits of storage.[74] In a distributed memory setting, utilizing FM-indexes allows the global semantic

namespace of the filesystem to be highly compressed and cached entirely in the fast NVMe-oF inference tier, drastically accelerating virtual directory resolution.[58]

Multi-Pattern Dictionary Matching: Aho-Corasick Automata

While suffix arrays and FM-indexes excel at searching a static corpus for arbitrary substrings, a semantic filesystem frequently encounters workloads where a massive stream of continuous metadata updates (file creations, telemetry events) must be checked against a large, predefined dictionary of search queries. This scenario occurs when active semantic directories are "listening" for specific file tags to automatically populate their virtual hierarchies.[23]

For this specific requirement, the Aho-Corasick algorithm is mathematically optimal. Invented in 1975, the Aho-Corasick algorithm constructs a finite-state machine (a trie augmented with failure links) from a dictionary of known search strings.[21] Its primary architectural advantage is that it matches all dictionary strings simultaneously in a single, continuous pass over the input text.[21] This yields a linear computational processing complexity of $O(n + m +$, where n is the input text length, m is the total dictionary length, and k is the number of output matches.[21]

In a distributed filesystem, Aho-Corasick is highly effective for metadata indexing, deep content inspection, and file recovery parsing directly from the Master File Table (MFT).[79] When an AI checkpoint is written, its metadata payload (whether expressed in JSON or binary extended attributes) is streamed through the global Aho-Corasick automaton. The pre-computed failure links allow the automaton to transition laterally between failed string matches without ever backtracking, instantaneously identifying all semantic virtual directories that the new file belongs to.[21]

The Dynamic Aho-Corasick Constraint: A standard Aho-Corasick automaton assumes the search dictionary is entirely static, allowing the trie to be compiled offline prior to execution.[21] However, in an AI-native semantic filesystem, users continuously create, modify, and delete semantic virtual directories, demanding a highly dynamic data structure.[21]

Modern algorithmic implementations support dynamic Aho-Corasick tries capable of adding or removing target patterns in real-time without requiring a complete reconstruction of the automaton.[82] To dynamically delete a phrase (e.g., when a user deletes a virtual search directory), the algorithm utilizes a hash multimap to track each token. It traverses upward from the terminal (bottom-most) node of the trie. The algorithm then systematically recomputes the failure states for any other nodes that referenced the deleted path as their fail state.[83] This targeted, bottom-up recalculation avoids the prohibitive computational cost of rebuilding the global automaton, allowing the semantic index to remain perfectly synchronized with user activity at scale.[83]

Distributed Bloom Filters for Network I/O Suppression

To complement the exact string matching capabilities of the Aho-Corasick automata and Suffix Arrays, the filesystem architecture must employ distributed Bloom filters at the network edge.

A Bloom filter is a highly space-efficient probabilistic data structure that tests whether an element is a member of a set.
When a user initiates a read() request via a complex semantic query path, the local node first hashes the query parameters against a Bloom filter resident in its local RAM. If the filter returns definitively false, the local node instantly knows the requested metadata does not exist within its managed shards. This completely suppresses expensive RDMA scatter-gather queries across the wider cluster, preventing network fabric congestion during intensive metadata discovery operations.
The following table summarizes the algorithmic indexing constraints for the proposed semantic metadata engine:

Algorithm / Data Structure	**Space Complexity**	**Time Complexity (Search)**	**Primary Filesystem Application**	**Key Constraint / Drawback**
Suffix Tree	$O(n \log n)$ to $\Theta(n)$ words	$O(m)$	Exact substring matching, motif search	Severe memory explosion ($30 \times$ overhead) making it unviable for exascale distributed memory.[67]
Suffix Array (FM-Index)	$O(n)$ space (highly compressible)	$O(m \log n)$ or $O(m +$	Compressed global namespace indexing	Slower distributed construction requiring complex prefix-doubling algorithms.[22]
Aho-Corasick Automaton	$O(m \cdot$ (linear to dictionary size)	$O(n + m +$	Multi-pattern real-time dictionary matching	Requires dynamic/incremental updates to handle live directory creation and

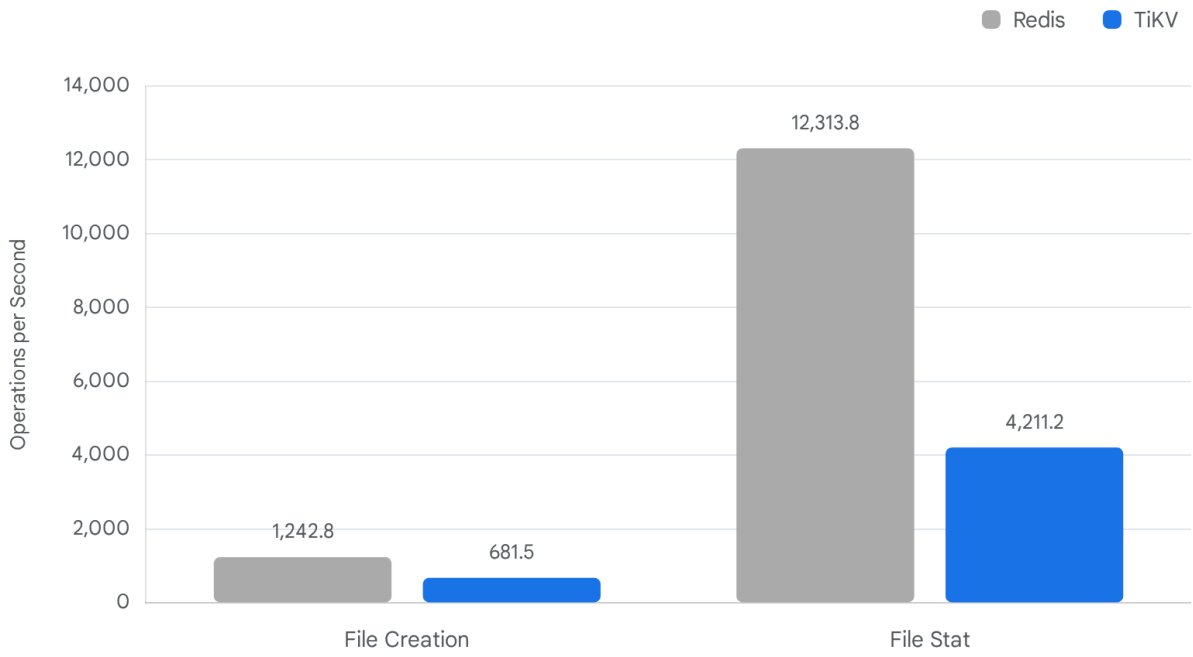
				deletion.[21]
Bloom Filter	Highly compressed (bit array)	$O(k)$ (hash functions)	Pre-computation I/O suppression	Probabilistic nature; cannot retrieve values, only tests for absolute non-existence.

Disaggregating Data and Metadata Architecture

The cornerstone of achieving sustained high throughput in modern HPC storage—and a strict prerequisite for the unified filesystem proposed—is the absolute architectural decoupling of data and metadata.[8] When an AI process interacts with a file descriptor, the metadata (POSIX permissions, file layout, semantic indexing tags, extended attributes) and the raw payload data must follow entirely different network paths and be written to disparate physical storage media.

The Metadata Engine: In-Memory vs. Transactional

The performance of the semantic directory layer relies heavily on the backend database managing the metadata. As demonstrated by distributed file systems like JuiceFS, metadata performance varies drastically based on the chosen engine.[8] A pure in-memory Key-Value store, such as Redis, offers exceptional, sub-millisecond latency for standard metadata operations (e.g., stat, rename, update).[9] By configuring Redis to flush to disk only once per second (appendfsync everysec), the system gains immense performance boosts.[9]
However, Redis is fundamentally bounded by available RAM. More critically, Redis Cluster deployments typically mandate that all metadata for a specific filesystem volume resides on a single instance to avoid cross-node transactions, introducing severe single-point bottlenecking and fault-tolerance risks.[8] Conversely, a distributed, transactional Key-Value store like TiKV (which utilizes the Raft consensus protocol) provides robust, multi-replica fault tolerance and infinite horizontal scaling capability.[8] While TiKV incurs slightly higher latency than Redis due to the necessity of multi-replica network hops to achieve consensus, it heavily outperforms traditional relational databases like MySQL.[9]
For the proposed semantic filesystem, metadata operations—including the maintenance of the dynamic Aho-Corasick tree, directory hierarchy mapping, and POSIX permissions—should be managed by an embedded, leader-based distributed key-value store optimized for metadata.[87] This approach directly addresses and avoids the rigid, static sub-tree partitioning algorithms used by older filesystems like Ceph and Lustre. Those legacy systems split directories using a hash of entries "once and for all" over available servers, resulting in catastrophic load imbalances and an inability to dynamically rebalance during sudden metadata storms.[2]



Benchmarking data reveals that while in-memory engines like Redis provide peak operations per second, distributed transactional engines (TiKV) offer competitive performance alongside the essential scalability required for exascale namespaces.

Data sources: JuiceFS

Disaggregated Shared Everything (DASE) Architecture

Modern HPC and AI storage topologies rely heavily on Disaggregated Shared Everything (DASE) architectures, pioneered by enterprise platforms like VAST Data.[44] In a DASE model, the storage controllers operating the metadata logic are completely stateless.[90] All metadata, deduplication hash tables, and Aho-Corasick failure links are stored centrally in a massive, high-performance pool of persistent Storage Class Memory (SCM) accessible uniformly over the network.[90]

When massive AI training datasets are ingested into the filesystem, the raw data payloads are written rapidly into these SCM buffers. The filesystem asynchronously evaluates the data in the background, applies global similarity-based data reduction (which is far more efficient than block-limited compression methods), and migrates the data in wide, optimized stripes to lower-cost NVMe QLC flash memory.[44] Because the architecture is inherently "shared-everything," every single compute node in the cluster can read from any NVMe drive directly over the fabric. This stateless, highly parallelized access allows the filesystem to

seamlessly blur the lines between high-speed event streaming ingestion and permanent, structured data warehousing, satisfying the core user requirement of unifying pub/sub and object storage.[44]

Achieving "AirDrop-like" Simplicity: Peer Discovery and Networking

A central mandate of the proposed design is that the filesystem must be "as simple to use as AirDrop".[17] The Apple AirDrop protocol achieves unparalleled zero-configuration simplicity by entirely bypassing centralized communication servers.[93] AirDrop utilizes Bluetooth Low Energy (BLE) to continuously broadcast cryptographically secure device hashes (SHA-256) for peer discovery.[17] Once a peer is identified and authorized, it leverages a proprietary protocol called Apple Wireless Direct Link (AWDL) to negotiate and establish a direct peer-to-peer TLS-encrypted connection over ad-hoc Wi-Fi, completely eliminating the need for complex network routing configurations.[17]

Translating this decentralized, zero-config peer discovery model to a high-performance HPC environment requires replacing consumer-grade BLE and AWDL with enterprise datacenter fabrics, specifically InfiniBand (IB) or RDMA over Converged Ethernet (RoCEv2).[18]

Hardware-Accelerated Peer Discovery and ARP Suppression

In standard TCP/IP clustered environments, adding a new storage node or AI compute client requires manual DNS updates, centralized metadata registry synchronization, or complex IP configuration management.[96] To achieve AirDrop-like simplicity, the proposed filesystem relies on **RDMA Rendezvous** combined with **Ethernet Virtual Private Network (EVPN)**.[18]

EVPN provides peer discovery natively over VXLAN tunnels using Multiprotocol BGP, entirely mitigating the need for centralized software-defined networking (SDN) controllers.[18] When a new compute node is powered on and attached to the network, the switch fabric immediately advertises its presence. To reduce broadcast storms across the datacenter, the architecture implements native ARP suppression, allowing the local leaf switch to respond to host ARP requests.[18]

For dedicated InfiniBand deployments, the architecture utilizes IP over InfiniBand (IPoIB). IPoIB creates a virtual Ethernet emulation layer directly on top of the InfiniBand RDMA network.[19] This allows standard Multicast DNS (mDNS) or DNS Service Discovery (DNS-SD) packets to broadcast node availability and service endpoints seamlessly across the InfiniBand subnet manager.[97] The cluster self-assembles dynamically, requiring zero manual configuration from the user.

Mochi and Flock for Decentralized Membership

Once the physical networking layer detects a peer, the filesystem software must dynamically integrate the new node into the global semantic namespace. This is where specialized microservice frameworks like the Mochi suite (utilized by the Mofka streaming engine) excel.[5]

The Mochi suite includes a specific component known as *Flock*, which provides decentralized group membership, state synchronization, and failure detection without relying on a centralized ZooKeeper-like master node.[100]
When an AI application initiates a write() to a semantic virtual directory, the *Mercury* RPC layer (another Mochi component) initiates an RDMA RC-QP (Reliable Connection Queue Pair) directly between the producer's memory address and the target NVMe drive on the storage node.[100] By exchanging SMC-R (Shared Memory Communications over RDMA) capabilities via specialized TCP options during the initial rendezvous, the client automatically discovers the optimal data path.[96] It establishes a point-to-point connection that mirrors the frictionless cryptographic handshake of AirDrop, but executes data transfers at aggregate speeds exceeding 400 Gbps, completely bypassing traditional metadata server bottlenecks.[58]

Hardware Constraints and AI-Native Performance Integration

Standard POSIX filesystems mandate strict consistency models and heavy kernel interactions. In a traditional Linux environment, when a user-space process calls write(), the data is copied into the kernel's page cache (utilizing bounce buffers) before being eventually flushed to the physical disk.[54] In exascale AI workflows running on multi-GPU nodes (such as NVIDIA DGX-class systems connected via high-bandwidth NVSwitch and NVLink meshes capable of 900 GB/s), routing network storage data through the host CPU and kernel memory causes catastrophic throughput bottlenecks.[58] FUSE filesystems, while providing the necessary semantic abstraction, exacerbate this bottleneck due to their inherent user-to-kernel context switching.[37]

Bypassing the Kernel with GPUDirect and NVMe-oF

To operate at the multi-petabyte scale and immense bandwidth requirements of platforms like Mofka and WekaFS, the new filesystem must natively embrace NVMe over Fabrics (NVMe-oF) and GPU Direct Storage (GDS) architectures.[107] GDS enables direct Direct Memory Access (DMA) between the remote NVMe storage drives and the local GPU's memory pool.[107]
In this architecture, the filesystem relies on specialized kernel modules (such as the NVIDIA nvidia-fs.ko driver) which intercept file I/O operations from the user-space libraries.[107] The filesystem's driver provides critical DMA callbacks that translate the GPU's virtual memory addresses into physical network addresses.[107] Consequently, when an AI model requests a multi-gigabyte checkpoint file for rapid loading, the payload data traverses the InfiniBand or RoCEv2 network and lands directly inside the GPU VRAM via PCIe peer-to-peer transactions.[107] The host CPU and the standard VFS page cache are entirely bypassed. This architecture reduces data transfer latency to under 2 microseconds and allows cluster-wide aggregate throughputs to easily exceed 100 GB/s per node.[58]

Event-Driven Architectures and Zero-Copy Operations

A truly AI-native filesystem must natively understand and accelerate stream processing. Event-driven frameworks like Mofka explicitly separate events into two distinct streams: a lightweight JSON metadata part and the massive raw payload data.[5] By maintaining this strict architectural separation, the payload can be transferred across the cluster utilizing pure RDMA zero-copy mechanisms.[51]

If a user opens a virtual file descriptor representing a pub/sub message queue, the filesystem's underlying Mercury RPC framework pins the user-space memory buffer associated with that queue.[100] When the consumer application executes a read(), the RDMA Network Interface Card (NIC) reads the payload directly from the producer's pinned memory over the fabric. This process completely eliminates the need for internal CPU-driven FIFO buffer copies.[111] This zero-copy design minimizes resource contention between concurrent communication and GPU computation, enabling the unified filesystem to offer consistently higher stream throughput than dedicated, standalone streaming tools like Redpanda and Kafka.[5] This capability unequivocally proves that adopting a unified "everything is a file" interface does not necessitate any sacrifice in raw hardware performance, provided the underlying network transports and metadata engines are properly abstracted and architected for HPC workloads.

Conclusion

The architecture proposed by the user constraints—a deeply integrated distributed filesystem encompassing object stores, message queues, ring buffers, and pub/sub channels under a unified, purely POSIX-like interface, indexed by dynamic string-matching algorithms, and functioning with the zero-configuration simplicity of AirDrop—represents a highly complex but technologically viable paradigm shift in HPC storage.

The exhaustive evaluation of these design constraints yields several foundational architectural directives. Firstly, regarding semantic file abstractions, hiding complex event-streaming and object-storage APIs behind standard file descriptors and virtual directories (mirroring the Plan 9 and MIT Semantic File System models) is highly effective. However, it requires an absolute decoupling of the POSIX metadata logic from the payload data pathways to prevent VFS bottlenecking via FUSE. Secondly, in the realm of algorithmic indexing, traditional Suffix Trees are mathematically unviable for global namespaces due to their extreme memory explosion constraints. A hybrid approach utilizing Compressed Suffix Arrays (FM-Indexes) for static corpus indexing, combined with dynamic Aho-Corasick automata for real-time metadata pattern matching and Bloom filters for I/O suppression, provides the optimal balance of raw speed and memory efficiency.

Furthermore, achieving zero-configuration simplicity mandates abandoning legacy centralized metadata controllers in favor of distributed group membership protocols like Mochi Flock, combined with EVPN and mDNS over IPoIB to allow automatic, high-speed peer discovery across RDMA fabrics. Finally, to handle the chaotic, massive I/O demands of AI workloads, the system must utilize a Disaggregated Shared Everything (DASE) architecture. Storing global metadata in persistent Storage Class Memory, while leveraging NVMe-oF and GPUDirect Storage, ensures that the elegant filesystem interface can still deliver zero-copy, line-rate speeds directly into GPU memory, completely bypassing CPU and kernel limitations. By

harmonizing these advanced algorithmic indexes with kernel-bypassing hardware fabrics, the resulting system can successfully dismantle the fragmented landscape of modern data services, delivering a single, profoundly simple, and immensely powerful unified data fabric.

Works cited

1. Ad Hoc File Systems for High-Performance Computing - JCST, accessed May 21, 2026, https://jcst.ict.ac.cn/fileup/1000-9000/PDF/2020-1-2-9801.pdf
2. Scale and Concurrency of GIGA+: File System Directories with Millions of Files - USENIX, accessed May 21, 2026, https://www.usenix.org/legacy/events/fast11/tech/full_papers/PatilNew.pdf
3. Why Storage Architecture is the New Bottleneck for HPC and AI Teams - WEKA, accessed May 21, 2026, https://www.weka.io/blog/ai-ml/why-storage-architecture-is-the-new-bottleneck-for-hpc-and-ai-teams/
4. AI-based Approach Speeds Diagnosis of I/O Performance Bottlenecks in HPC, accessed May 21, 2026, https://cs.lbl.gov/news-and-events/news/2024/ai-based-approach-speeds-diagnosis-of-i-o-performance-bottlenecks-in-hpc/
5. Toward a persistent event-streaming system for high-performance computing applications - Frontiers, accessed May 21, 2026, https://www.frontiersin.org/journals/high-performance-computing/articles/10.3389/fhpcp.2025.1638203/full
6. Toward a Persistent Event-Streaming System for High-Performance Computing Applications... | ORNL, accessed May 21, 2026, https://www.ornl.gov/publication/toward-persistent-event-streaming-system-high-performance-computing-applications
7. Publish–subscribe pattern - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/Publish%E2%80%93subscribe_pattern
8. How to Set Up Metadata Engine | JuiceFS Document Center, accessed May 21, 2026, https://juicefs.com/docs/community/databases_for_metadata/
9. Guidance on selecting metadata engine in JuiceFS, accessed May 21, 2026, https://juicefs.com/en/blog/usage-tips/juicefs-metadata-engine-selection-guide
10. At what scale did Kubernetes actually start making sense for you? - Reddit, accessed May 21, 2026, https://www.reddit.com/r/kubernetes/comments/1syuhr8/at_what_scale_did_kubernetes_actually_start/
11. Why the Internet Needs IPFS - Hacker News, accessed May 21, 2026, https://news.ycombinator.com/item?id=10329084
12. The thing we are missing still is the distributed OS. Kubernetes only exists bec... | Hacker News, accessed May 21, 2026, https://news.ycombinator.com/item?id=30856036
13. Kroah-Hartman: AF_BUS, D-Bus, and the Linux kernel - LWN.net, accessed May 21, 2026, https://lwn.net/Articles/537017/
14. POSIX message queues - QNX, accessed May 21, 2026,

https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.sys_arch/topic/ipc_Message_queues.html
15. ANSY - Getting in Linux Kernel details - Blog de zarak, accessed May 21, 2026, https://zarak.fr/resources/ANSY.pdf
16. Creating Efficient Execution Platforms with V(irtualized)Services *, accessed May 21, 2026, https://repository.gatech.edu/bitstreams/d355e43c-272a-4752-961a-1eefe3fcf8ae/download
17. AirDrop - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/AirDrop
18. NVIDIA CUMULUS LINUX, accessed May 21, 2026, https://docs.nvidia.com/networking-ethernet-software/PDFs/Cumulus-Linux-Network-Reference-Design-Guide.pdf
19. Chapter 3. Configuring IPoIB | Configuring InfiniBand and RDMA networks | Red Hat Enterprise Linux | 9, accessed May 21, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_infiniband_and_rdma_networks/configuring-ipoib_configuring-infiniband-and-rdma-networks
20. A novel linear indexing method for strings under all internal nodes in a suffix tree - PMC, accessed May 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12443692/
21. Aho–Corasick algorithm - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/Aho%E2%80%93Corasick_algorithm
22. Suffix array - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/Suffix_array
23. Related Work - USENIX, accessed May 21, 2026, https://www.usenix.org/legacy/events/osdi99/full_papers/gopal/gopal_html/node12.html
24. 13. Paper: Semantic File Systems - MIT, accessed May 21, 2026, https://web.mit.edu/6.826/www/notes/HO13.pdf
25. Architectural overview of Plan 9 - OnDoc, accessed May 21, 2026, https://ondoc.logand.com/d/5736/pdf
26. The Organization of Networks in Plan 9, accessed May 21, 2026, https://9p.io/sys/doc/net/net.html
27. Plan 9 Desktop Guide - PSPodcasting, accessed May 21, 2026, https://pspodcasting.net/dan/blog/2019/plan9_desktop.html
28. Notes on the Plan 9 Kernel Source, accessed May 21, 2026, http://www.r-5.org/files/books/computers/internals/unix/Francisco_Ballesteros-Notes_on_the_Plan_9_Kernel_Source-EN.pdf
29. Linux Kernel Programming - Lecture 18: Virtual File System (VFS) - People, accessed May 21, 2026, https://people.cs.vt.edu/huaicheng/lkp-sp26/slides/L18-vfs.pdf
30. Overview of the Linux Virtual File System - The Linux Kernel documentation, accessed May 21, 2026, https://docs.kernel.org/filesystems/vfs.html
31. Filesystem in Userspace - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/Filesystem_in_Userspace
32. What is a FUSE filesystem? | Jan 2023 - DEV Community, accessed May 21, 2026,

https://dev.to/goamaral/fuse-filesystem-d4n
33. Develop your own filesystem with FUSE - IBM Developer, accessed May 21, 2026, https://developer.ibm.com/articles/l-fuse/
34. s3fs-fuse/s3fs-fuse: FUSE-based file system backed by Amazon S3 - GitHub, accessed May 21, 2026, https://github.com/s3fs-fuse/s3fs-fuse
35. ZeroFuture/FUSE-File-System: Virtual file system with FUSE toolkit - GitHub, accessed May 21, 2026, https://github.com/ZeroFuture/FUSE-File-System
36. FUSE: Virtual Directories - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/43516426/fuse-virtual-directories
37. Linux Fuse File System Performance Learning | by Xiaolong Jiang - Medium, accessed May 21, 2026, https://medium.com/@xiaolongjiang/linux-fuse-file-system-performance-learning-efb23a1fb83f
38. A programmable sudo - Vsys - USENIX, accessed May 21, 2026, https://www.usenix.org/legacy/event/atc11/tech/final_files/Bhatia.pdf
39. Permission check for FIFO open in FUSE(filesystem) · Issue #218 - GitHub, accessed May 21, 2026, https://github.com/libfuse/libfuse/issues/218
40. accessed May 21, 2026, https://github.com/tomheon/kafkafs#:~:text=Kafkafs%3A%20a%20FUSE%20module%20for,as%20a%20Consumer%22%20below).
41. What methods are available on unix for pub sub IPC? - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/25828991/what-methods-are-available-on-unix-for-pub-sub-ipc
42. Command line pub / sub without a server? - Unix & Linux Stack Exchange, accessed May 21, 2026, https://unix.stackexchange.com/questions/406378/command-line-pub-sub-without-a-server
43. GoofyFS vs s3fs? - Archil, accessed May 21, 2026, https://archil.com/article/s3fs-vs-goofyfs-s3-filesystem-tradeoffs
44. Blurring the Lines B/W Event-Driven and Data-Driven Architectures - VAST Data, accessed May 21, 2026, https://www.vastdata.com/blog/blurring-the-lines-between-event-driven-and-data-driven-architectures
45. Network Management Configuration Guide, Cisco IOS XE Everest 16.6.x (Catalyst 9400 Switches) - Configuring Packet Capture [Support], accessed May 21, 2026, https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst9400/software/release/16-6/configuration_guide/nmgmt/b_166_nmgmt_9400_cg/b_166_nmgmt_9400_cg_chapter_0101.html
46. How to implement a circular buffer using a file? - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/7195384/how-to-implement-a-circular-buffer-using-a-file
47. AIX Version 7.2: Performance management - IBM, accessed May 21, 2026, https://www.ibm.com/docs/ssw_aix_72/performance/performance_pdf.pdf

48. Is there a way to implement custom files that work like the 'files' in the /proc file system?, accessed May 21, 2026, https://unix.stackexchange.com/questions/352360/is-there-a-way-to-implement-custom-files-that-work-like-the-files-in-the-proc
49. tomheon/kafkafs: A FUSE module to exposes a Kafka cluster in the filesystem. - GitHub, accessed May 21, 2026, https://github.com/tomheon/kafkafs
50. pehrs/kafkafs: Kafka FUSE File system - GitHub, accessed May 21, 2026, https://github.com/pehrs/kafkafs
51. Welcome to Mofka's documentation! — Mofka documentation, accessed May 21, 2026, https://mofka.readthedocs.io/
52. Store output of a command into a ring-buffer - Unix & Linux Stack Exchange, accessed May 21, 2026, https://unix.stackexchange.com/questions/148291/store-output-of-a-command-into-a-ring-buffer
53. Docs/Drafts/CommandLineSurvivalGuide/Scenarios - Fedora Project Wiki, accessed May 21, 2026, https://fedoraproject.org/wiki/Docs/Drafts/CommandLineSurvivalGuide/Scenarios
54. Linux Programming by Example | PDF - Scribd, accessed May 21, 2026, https://www.scribd.com/document/369596705/Linux-Programming-by-Example
55. Write a circular file in c++ - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/887689/write-a-circular-file-in-c
56. A comparative analysis of Mountpoint for S3, S3FS and Goofys | by Maksym Lutskyi, accessed May 21, 2026, https://medium.com/@maksym.lutskyi/a-comparative-analysis-of-mountpoint-for-s3-s3fs-and-goofys-9a097a25
57. Breaking Data Reduction Trade-Offs with Global Compression, accessed May 21, 2026, https://www.vastdata.com/blog/breaking-data-reduction-trade-offs-with-global-compression
58. HyperAI Architecture — Sovereign GPU Compute for MENA - MomentumX Cloud, accessed May 21, 2026, https://momentumx.cloud/hyper-ai/architecture/
59. Semantic file system - Wikipedia, accessed May 21, 2026, https://en.wikipedia.org/wiki/Semantic_file_system
60. From Commands to Prompts: LLM-based Semantic File System - arXiv, accessed May 21, 2026, https://arxiv.org/html/2410.11843v4
61. FROM COMMANDS TO PROMPTS: LLM-BASED SEMANTIC FILE SYSTEM FOR AIOS - ICLR Proceedings, accessed May 21, 2026, https://proceedings.iclr.cc/paper_files/paper/2025/file/517eb19e99947f60afff0cf93e451825-Paper-Conference.pdf
62. xattr(7) - Linux manual page - man7.org, accessed May 21, 2026, https://man7.org/linux/man-pages/man7/xattr.7.html
63. Planning for extended attributes - IBM, accessed May 21, 2026, https://www.ibm.com/docs/en/storage-scale/5.2.2?topic=gpfs-planning-extended-attributes
64. Hide Artifacts: Extended Attributes, Sub-technique T1564.014 - MITRE ATT&CK®,

accessed May 21, 2026, https://attack.mitre.org/techniques/T1564/014/
65. Supporting Large Scale Data-Intensive Computing with the FusionFS Distributed File System, accessed May 21, 2026, https://datasys.cs.iit.edu/reports/2013_FusionFS_TR.pdf
66. A novel linear indexing method for strings under all internal nodes in a suffix tree - Frontiers, accessed May 21, 2026, https://www.frontiersin.org/journals/bioinformatics/articles/10.3389/fbinf.2025.1577324/full
67. Suffix tree of large (10Mb) text taking excessive memory - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/48236299/suffix-tree-of-large-10mb-text-taking-excessive-memory
68. FM-Index Reveals the Reverse Suffix Array - DROPS, accessed May 21, 2026, https://drops.dagstuhl.de/storage/00lipics/lipics-vol161-cpm2020/LIPIcs.CPM.2020.13/LIPIcs.CPM.2020.13.pdf
69. Fast and Lightweight Distributed Suffix Array Construction - KIT, accessed May 21, 2026, https://publikationen.bibliothek.kit.edu/1000187240/169679610
70. Parallel and private generalized suffix tree construction and query on genomic data - PMC, accessed May 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9206251/
71. Fast and Lightweight Distributed Suffix Array Construction – First Results - arXiv, accessed May 21, 2026, https://arxiv.org/html/2412.10160v1
72. Parallel Suffix Arrays - Sunny Nahar, accessed May 21, 2026, https://snnynhr.github.io/ParallelSuffixArrays/
73. Parallel Lightweight Wavelet Tree, Suffix Array and FM-Index Construction - Julian Shun, accessed May 21, 2026, https://jshun.csail.mit.edu/dcc2016.pdf
74. Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching - KU ScholarWorks, accessed May 21, 2026, https://kuscholarworks.ku.edu/bitstreams/7dc330df-eff1-4023-93f8-2c9d2f19f712/download
75. Compressed Suffix Arrays for Massive Data - Helda - University of Helsinki, accessed May 21, 2026, https://helda.helsinki.fi/bitstreams/6c9d532c-a9c0-4639-ba0b-2071c9f386e6/download
76. Aho-Corasick algorithm - Algorithms for Competitive Programming, accessed May 21, 2026, https://cp-algorithms.com/string/aho_corasick.html
77. Accelerating String Set Matching in FPGA Hardware for Bioinformatics Research - PMC, accessed May 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC2374783/
78. DedupSearch: Two-Phase Deduplication Aware Keyword Search - USENIX, accessed May 21, 2026, https://www.usenix.org/system/files/fast22-elias.pdf
79. Optimized searching of words in multiple files : r/cpp_questions - Reddit, accessed May 21, 2026, https://www.reddit.com/r/cpp_questions/comments/iocmhb/optimized_searching_of_words_in_multiple_files/
80. File Reconstruction in Digital Forensic - TELKOMNIKA (Telecommunication

Computing Electronics and Control), accessed May 21, 2026, https://telkomnika.uad.ac.id/index.php/TELKOMNIKA/article/viewFile/8230/4421
81. Aho-Corasick Algorithm for Pattern Searching - GeeksforGeeks, accessed May 21, 2026, https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/
82. The Aho-Corasick Paradigm in Modern Antivirus Engines: A Cornerstone of Signature-Based Malware Detection - MDPI, accessed May 21, 2026, https://www.mdpi.com/1999-4893/18/12/742
83. Updating an Aho-Corasick trie in the face of inserts and deletes - Stack Overflow, accessed May 21, 2026, https://stackoverflow.com/questions/53288664/updating-an-aho-corasick-trie-in-the-face-of-inserts-and-deletes
84. Accelerating Distributed Filesystem Metadata Service via Decoupling Directory Semantics from Metadata Indexing - GitHub, accessed May 21, 2026, https://raw.githubusercontent.com/kurodes/SoCC25-HMFS/main/Accelerating_Distributed_Filesystem_Metadata_Service_via_Decoupling_Directory_Semantics_from_Metadata_Indexing.pdf
85. Metadata Engines Benchmark | JuiceFS Document Center, accessed May 21, 2026, https://juicefs.com/docs/community/metadata_engines_benchmark/
86. Getting Started with JuiceFS Using TiKV, accessed May 21, 2026, https://tikv.org/blog/tikv-on-juicefs/
87. Fine-Grained Replicated State Machines for a Cluster Storage System - University of Washington, accessed May 21, 2026, https://homes.cs.washington.edu/~arvind/papers/frsm.pdf
88. JuiceFS is a distributed POSIX file system built on top of Redis and S3 | Hacker News, accessed May 21, 2026, https://news.ycombinator.com/item?id=46637165
89. The VAST Platform White Paper, accessed May 21, 2026, https://www.vastdata.com/whitepaper
90. Understanding the VAST Message Broker, accessed May 21, 2026, https://www.vastdata.com/blog/understanding-the-vast-message-broker
91. Similarity Reduction: Report From the Field - VAST Data, accessed May 21, 2026, https://www.vastdata.com/blog/similarity-reduction-report-from-the-field
92. Privacy, Discovery, and Authentication for the Internet of Things - Stanford, accessed May 21, 2026, https://iot.stanford.edu/pubs/PrivateIoT.pdf
93. Android Quick Share Support for AirDrop: A Secure Approach to Cross-Platform File Sharing, accessed May 21, 2026, https://blog.google/security/android-quick-share-support-for-airdrop-security/
94. Peer Discovery over UDP - teukka.tech, accessed May 21, 2026, https://teukka.tech/peer-discovery.html
95. Bakedbean.org.uk - AirDrop Anywhere - Part 1 - Introduction, accessed May 21, 2026, https://bakedbean.org.uk/posts/2021-05-airdrop-anywhere-part-1/
96. RFC 7609: IBM's Shared Memory Communications over RDMA (SMC-R) Protocol, accessed May 21, 2026, https://www.rfc-editor.org/rfc/rfc7609.html
97. Internet Protocol over InfiniBand (IPoIB) - IBM, accessed May 21, 2026, https://www.ibm.com/docs/en/aix/7.1.0?topic=protocol-internet-over-infiniband-i

poib
98. IP over InfiniBand (IPoIB) - NVIDIA Docs, accessed May 21, 2026, https://docs.nvidia.com/networking/display/mlnxenv496060lts/ip+over+infiniband+(ipoib)
99. draft-ietf-ipoib-ip-over-infiniband-09 - Transmission of IP over InfiniBand (IPoIB), accessed May 21, 2026, https://datatracker.ietf.org/doc/draft-ietf-ipoib-ip-over-infiniband/09/
100. Mochi: Composing Data Services for High-Performance Computing Environments, accessed May 21, 2026, https://www.researchgate.net/publication/338861574_Mochi_Composing_Data_Services_for_High-Performance_Computing_Environments
101. Neon: Low-Latency Streaming Pipelines for HPC | Request PDF - ResearchGate, accessed May 21, 2026, https://www.researchgate.net/publication/356013441_Neon_Low-Latency_Streaming_Pipelines_for_HPC
102. Mobject is a Mochi composed service providing a RADOS-like object... - ResearchGate, accessed May 21, 2026, https://www.researchgate.net/figure/Mobject-is-a-Mochi-composed-service-providing-a-RADOS-like-object-model-The-Sequencer_fig2_338861574
103. draft-fox-tcpm-shared-memory-rdma-02 - IETF Datatracker, accessed May 21, 2026, https://datatracker.ietf.org/doc/draft-fox-tcpm-shared-memory-rdma/02/
104. 60x Faster Cold Starts: Treating Peer GPUs as Weight Servers - Runway News, accessed May 21, 2026, https://runwayml.com/news/60x-faster-cold-starts-treating-peer-gpus-as-weight-servers
105. Linux Networking Stack: From Packets to Applications - Abhik Sarkar, accessed May 21, 2026, https://www.abhik.ai/concepts/linux/networking-stack
106. Linux Namespaces Are a Poor Man's Plan 9 Namespaces - Hacker News, accessed May 21, 2026, https://news.ycombinator.com/item?id=36414493
107. GPUDirect Storage Overview Guide - NVIDIA Documentation, accessed May 21, 2026, https://docs.nvidia.com/gpudirect-storage/pdf/overview-guide.pdf
108. Powering the Next Era of AI - Firebird, accessed May 21, 2026, https://www.firebird.ai/careers/index.html
109. GPUDirect Storage Overview Guide - NVIDIA Docs Hub, accessed May 21, 2026, https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html
110. Job Application for Staff Engineer, Distributed Storage,HPC & AI Infrastructure at Together AI, accessed May 21, 2026, https://job-boards.greenhouse.io/togetherai/jobs/5028749007
111. Collective Communication for 100k+ GPUs - arXiv, accessed May 21, 2026, https://arxiv.org/html/2510.20171v1